

Image Processing

Image Processing Basics

By

Dr. Rasha Mahmoud Kamel



Digital Image Representation–Image Layout

- A **digital image** can be considered as *a discrete representation of data* possessing both spatial (layout) and intensity (color) information.
- A **digital image** can be *represented as a two-dimensional (2D) matrix* of real numbers.
- Suppose that I represents a 2D discrete digital image of size $M \times N$ and $I(m,n)$ represents the intensity or color at a series of pixel positions $(m = 1, 2, \dots, M ; n = 1, 2, \dots, N)$ in 2D Cartesian coordinates.
- The indices m and n designate the rows and columns of the image, respectively.
- The individual *picture elements or pixels* of the image are thus referred to by their 2D (m, n) index.

Digital Image Representation–Image Layout

- Following the **MATLAB** convention, *monochrome images are essentially 2D matrices* (or arrays) where $I(m,n)$ denotes the intensity or gray level of the image at the pixel located at the m^{th} row and n^{th} column starting from *a top-left image origin*.



Digital Image Representation–Image Layout

- **MATLAB** and its Image Processing Toolbox (IPT) use *1-based array notation*.
- A monochrome image matrix looks like this:

$$I = \begin{bmatrix} I(1,1) & I(1,2) & \cdots & I(1,N) \\ I(2,1) & I(2,2) & \cdots & I(2,N) \\ \vdots & \vdots & \cdots & \vdots \\ I(M,1) & I(M,2) & \cdots & I(M,N) \end{bmatrix}$$

Digital Image Representation–Image Color

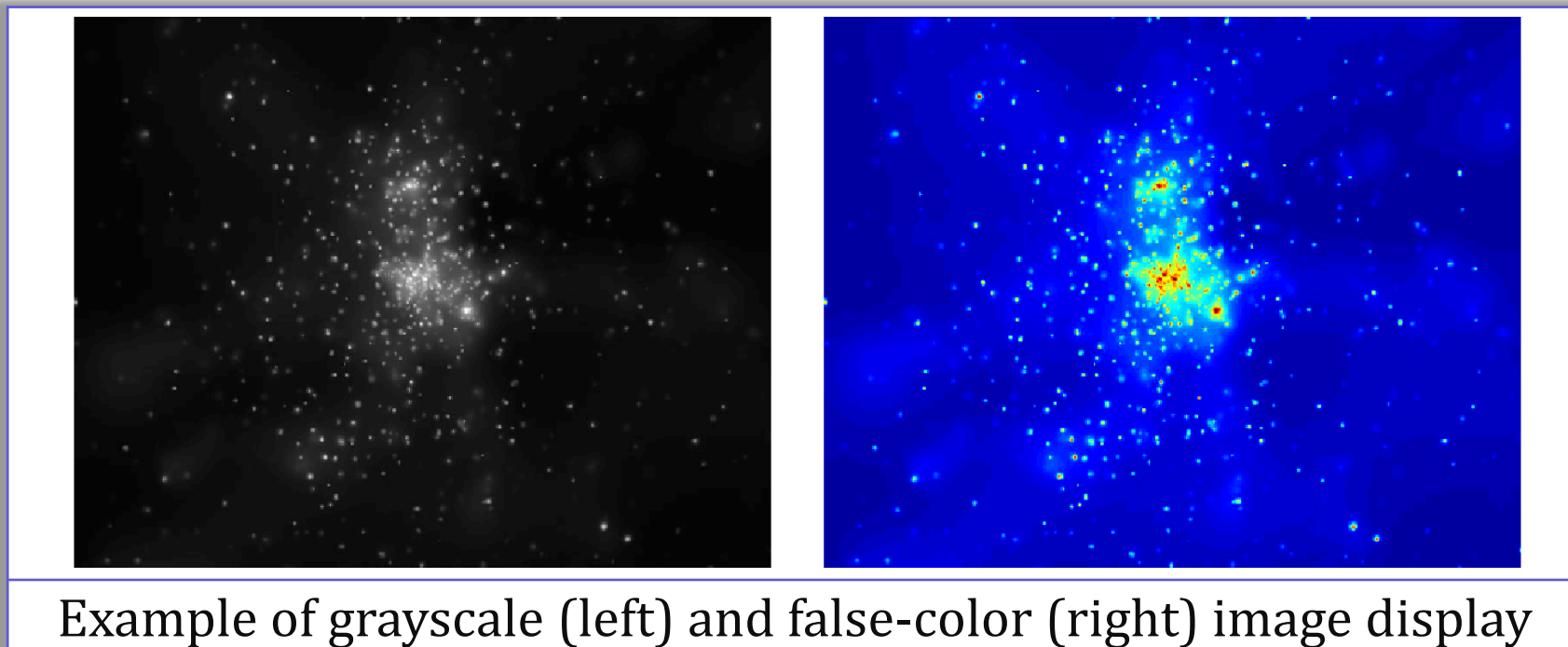
- **An image** *contains one or more color channels* that define the intensity or color at each pixel location $I(m, n)$.
- In the simplest case, *each pixel location only contains a single numerical value* representing the signal level at that point in the image.
- To visualize these values, a **color map** is used, which *assigns a specific shade of color to each numerical level in the image* to give a visual representation of the data.
- The most common *color map is the greyscale, which is particularly well suited for intensity images*, where each pixel expresses the intensity of the signal as a single value.

Digital Image Representation–Image Color

- **The greyscale color map** *assigns all shades of grey from black (zero) to white (maximum)* according to the signal level.
- Sometimes, it can be better to display *intensity images using a false-color map*, which assigns colors to different data values. A key reason for using false-color maps is that the human visual system can only distinguish about 40 shades of grey between black and white, whereas our sensitivity to color is much finer.
- False-color maps can also *highlight specific features or structures*, making them easier to identify for the human observer. This technique is commonly used in medical and astronomical imaging.
- MATLAB provides a variety of color maps to choose from, including jet, hot, cool, and parula.

Digital Image Representation–Image Color

- This figure shows an astronomical intensity image displayed using both greyscale and a particular false-color map. In this example, the *jet color map* has been used to *highlight the structure and finer detail of the image* to the human viewer using a linear color scale ranging from dark blue (low intensity values) to dark red (high intensity values).



Example of grayscale (left) and false-color (right) image display

Digital Image Representation–Image Color

- In addition to greyscale images, where we have a single numerical value at each pixel location, we also have *truecolor images* where the full spectrum of *colors can be represented as a triplet vector*, typically the (R,G,B) components at each pixel location. Here, the color is represented as a linear combination of the basis colors or values, and the image may be considered as consisting of three 2D planes.
- The Hue, Saturation, and Value (HSV) is an alternative way of representing truecolor images that is used quite widely. In this representation, H is the dominant wavelength of the color, e.g. red, blue, green. S is the purity of color (in the sense of the amount of white light mixed with it). V is the brightness of the color (also known as luminance).

Resolution and Quantization

- The **spatial resolution** of an image refers to *the number of pixels in an image*, which is determined by the size of the 2D pixel grid (width $\times$ height). Higher resolution means more pixels, leading to finer details and sharper images.
- The **color quantization** (color depth) of an image refers to *the data size stored for each pixel*. This determines how many colors a pixel can represent, which is usually measured in bits per pixel (bpp). More bits per pixel generates more colors.
- The resolution of an image source (e.g. a camera) can be specified in terms of three quantities:
 - (1) **Spatial resolution:** This defines *the number of pixels in the captured image*. This is defined by the dimensions of the image ($M \times N$) where M represents the number of pixels in height (rows) and N represents the number of pixels in width (columns). This is sometimes referred to as the pixel or digital resolution of the image.

Resolution and Quantization

- (2) **Temporal resolution:** For a continuous capture system such as video, it refers to *how many frames per second* (FPS) a camera can capture. Higher FPS means smoother motion in videos.
- (3) **Bit resolution (Bit depth):** It refers to *the number of bits used to represent the color* or grayscale value of each pixel in an image. This defines the number of possible intensity/color values that a pixel may have. A *binary image* requires 1 bit per pixel so it has just two colors (black or white). A *greyscale image* requires 8 bit per pixel, allowing it to have 256 different gray levels ranging from black to white. A *color image* commonly uses 8 bits per color channel ($8 \text{ bits} \times 3 \text{ channels} = 24 \text{ bits per pixel}$), allowing it to represent 16.7 million colors. The range of possible values a pixel can take is often referred to as the *dynamic range of an image* (e.g., an 8-bit grayscale image has 256 intensity levels (0–255)).

Image Formats

- In the real world, we need to effectively display images, store them, transmit them over networks, etc. This has led to the development of standard digital image formats.
- **Image file format** is a *standard way to organize and store image data*. It defines how the data is arranged and the type of compression—if any—that is used.
- **The image formats** comprise a *file header and the actual numeric pixel values* themselves.
- The image **file header** *stores information about the image* such as image height and width, number of bits per pixel, etc.

Image Formats

- There are a large number of recognized image formats now existing. Common image formats and their associated properties are listed in the following table.

Acronym	Name	Properties
GIF	Graphics interchange format	Limited to only 256 colors (8 bit); lossless compression
JPEG	Joint Photographic Experts Group	In most common use today; lossy compression
BMP	Bit map picture	Basic image format; lossless compression
PNG	Portable network graphics	New lossless compression format
TIF/TIFF	Tagged image (file) format	Highly flexible, detailed and adaptable format; both lossy and lossless compression

Image Compression

- Raw image representations refer to uncompressed image data, meaning they store all pixel values without compression. This results in large file sizes and consequently longer transmission times when uploading or downloading files.
- **Compressing an image** reduces its file size, requiring less disk storage and enabling faster transfer over a network.
- *Most image file formats employ some type of compression.* Thus, the main consideration in choosing an image storage format is compression.
- **Compression methods** can be classified into *lossy compression and lossless compression* techniques.

Image Compression – Lossy Compression

- **Lossy compression** reduces file size by *removing redundant* or less perceptible *information from the image*.
- This may involve discarding fine image details or *reducing the number of colors/grey levels*.
- This can *significantly reduce the amount of image information* that needs to be stored, leading to *a significant reduction in image quality*.



Original Image (8-bit RGB)
= 1024 x 768 x 3
= 2304Kb $\approx$ 2.3Mb



JPEG (Quality : 0) = 16k



JPEG (Quality : 20) = 40k



JPEG (Quality : 75) = 168k

An example for the image compressed using varying levels of lossy compression

Image Compression – Lossy Compression

- This process is irreversible, meaning that once compressed, the original data is lost.
- Some image formats store data using lossy compression, such as the JPEG format.
- **Lossy compression** should be used for general-purpose photographic images or *images in which loss of detail may be acceptable.*



Original Image (8-bit RGB)
= 1024 x 768 x 3
= 2304Kb $\approx$ 2.3Mb



JPEG (Quality : 0) = 16k



JPEG (Quality : 20) = 40k



JPEG (Quality : 75) = 168k

An example for the image compressed using varying levels of lossy compression

Image Compression – Lossless Compression

- In contrast to lossy compression, **lossless compression technique** reduces file size without losing any image data.
- Thus, the original image can be *perfectly reconstructed* after decompression.
- It is used in common image formats such as PNG format.
- **Lossless compression** should be preferred when dealing with *images in which loss of detail may be unacceptable* such as space images and medical images, faxes, etc.



Original Image (8-bit RGB)
= 1024 x 768 x 3
= 2304Kb ~ 2.3Mb



PNG (max. compression) = 1.4Mb

An example for the image that is compressed using lossless compression technique

Image Compression

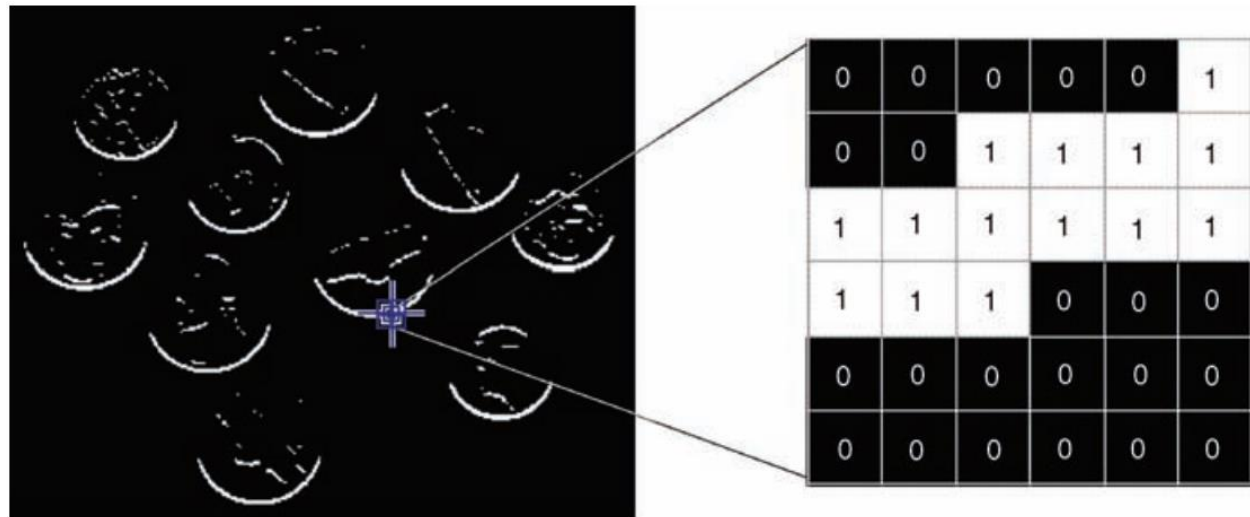
- In terms of practical image processing in **MATLAB**, it should be noted that an image written to file from MATLAB in *a lossy compression format* (e.g. JPEG) will not be stored as the exact MATLAB image representation it started as. *Image pixel values will be altered* in the image output process as a result of the lossy compression. *This is not the case if a lossless compression technique is employed.*

Image Data Types – Binary Images

- **Binary images** are 2D arrays that assign one numerical value from the set $\{0,1\}$ to each pixel in the image. These are sometimes referred to as logical images. Black corresponds to zero (an 'off' or 'background' pixel) and white corresponds to one (an 'on' or 'foreground' pixel).
- Binary images are typically represented in **MATLAB** using a logical array of 0's and 1's.



Image: binary
Pixel Data Type: integer (0 or 1)
Image Format: PNG



Binary image and the pixel values in a 6×6 neighborhood

Image Data Types – Greyscale Images

- **Intensity or greyscale images** are 2D arrays that assign one numerical value to each pixel which is representative of the intensity at this point.
- The pixel value range is bounded by the bit resolution of the image and such images are stored as N-bit integer images.
- Each pixel can be assigned whole integers in the range of $[0 \dots 2^k - 1]$.
- A typical grayscale image uses $k = 8$ bits (1 byte) per pixel and intensity values in the range of $[0 \dots 255]$, where the value 0 corresponds to “black”, 255 means “white”, and intermediate values indicate varying shades of gray.

Image Data Types – Greyscale Images

- Intensity images can be represented in **MATLAB** using different data types (uint8, uint16, double). For greyscale images with elements of integer classes uint8 and uint16, each pixel has a value in the $[0, 255]$ and the $[0, 65,535]$ range, respectively. Greyscale images of class double have pixel values in the $[0.0, 1.0]$ range.



Image: 8-bit grayscale
Pixel Data Type: integer (0→255)
Image Format: GIF

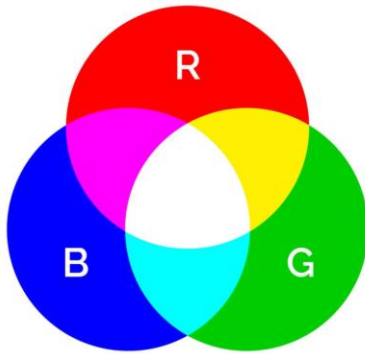


255	255	255	255	255	195
255	255	255	242	129	50
255	255	185	61	68	110
255	133	42	86	109	110
112	56	99	107	98	109
66	98	98	97	109	104

A grayscale image and the pixel values in a 6×6 neighborhood

Image Data Types – Color Images

- Most **color images** are based on the primary colors *red, green, and blue (RGB)*.



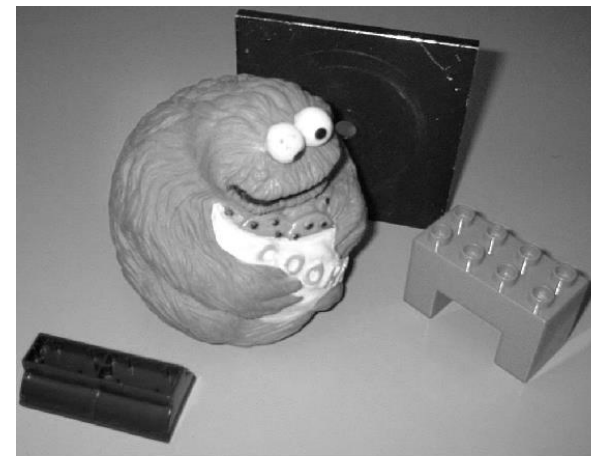
- Color images can be represented using *three 2D arrays of same size*, one for each color channel: R, G, and B.



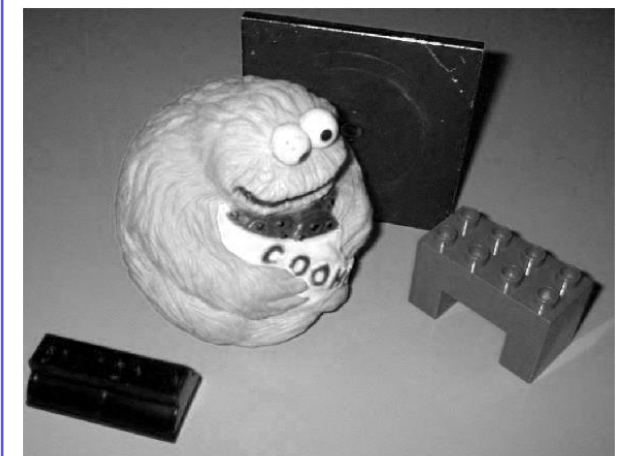
Original



Red Channel



Green Channel



Blue Channel

Color image and its red, green, and blue components

Image Data Types – Color Images

- **Each array** element contains *an 8-bit value*, indicating the amount of red, green, or blue at that point.
- *Each pixel requires* $3 \times 8 = 24$ bits to encode all three components, and the range of each individual color component is $[0 \dots 255]$.
- The combination of the three 8-bit values into a *24-bit number allows* 2^{24} (16,777,216) color combinations.

62	63	63	65	66					
63	61	59	64	63					
65	63	63	66	66					
63	67	29	31	34	30	27			
63	62	31	31	32	30	28			
		29	30	31	30	31			
		29	29	64	64	64	60	59	
		31	32	62	64	64	60	59	
				60	62	63	61	61	
				62	63	63	60	62	
				62	63	62	61	61	



Image Data Types – Color Images

- RGB color images *are represented in MATLAB as 3D arrays* of dimension $(M \times N \times 3)$, where M denotes the number of image rows and N the number of image columns.
- Commonly, such images are then *accessed in MATLAB by* $I(M, N, channel)$ coordinates within the 3D array.

62	63	63	65	66					
63	61	59	64	63					
65	63	63	66	66					
63	67	29	31	34	30	27			
63	62	31	31	32	30	28			
		29	30	31	30	31			
		29	29	64	64	64	60	59	
		31	32	62	64	64	60	59	
				60	62	63	61	61	
				62	63	63	60	62	
				62	63	62	61	61	



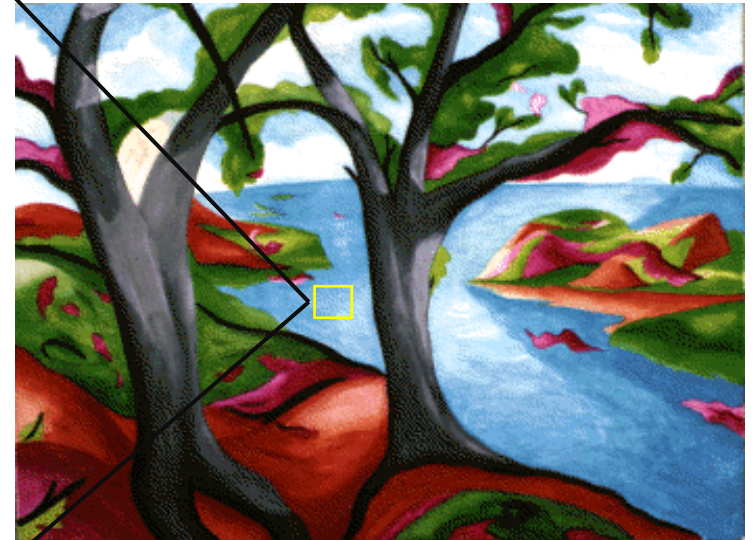
Image Data Types – Indexed Color Images

- **Indexed color images** are a unique type of color image. The difference between an *indexed image and a truecolor image* is that an indexed image has a limited number of colors (typically 256 or fewer), while a truecolor image can display millions of colors.
- **In an indexed image**, *the pixel values are only indices (pointers)* with a maximum of 8 bits *to a color palette (or color map)*.
- **The color map** *is simply a list of colors* C of fixed maximum size (usually $C = 256$ colors) *that are used in the image*.
- **The color map** is an $C \times 3$ *array of class double containing values in the range* $[0.0, 1.0]$. Each row of the color map specifies the red, green, and blue components of a single color.

Image Data Types – Indexed Color Images

- *The pixel values of an indexed image must be integers* in the range $[0, C]$. The value 0 points to the first row in color map, the value 1 points to the second row, and so on.

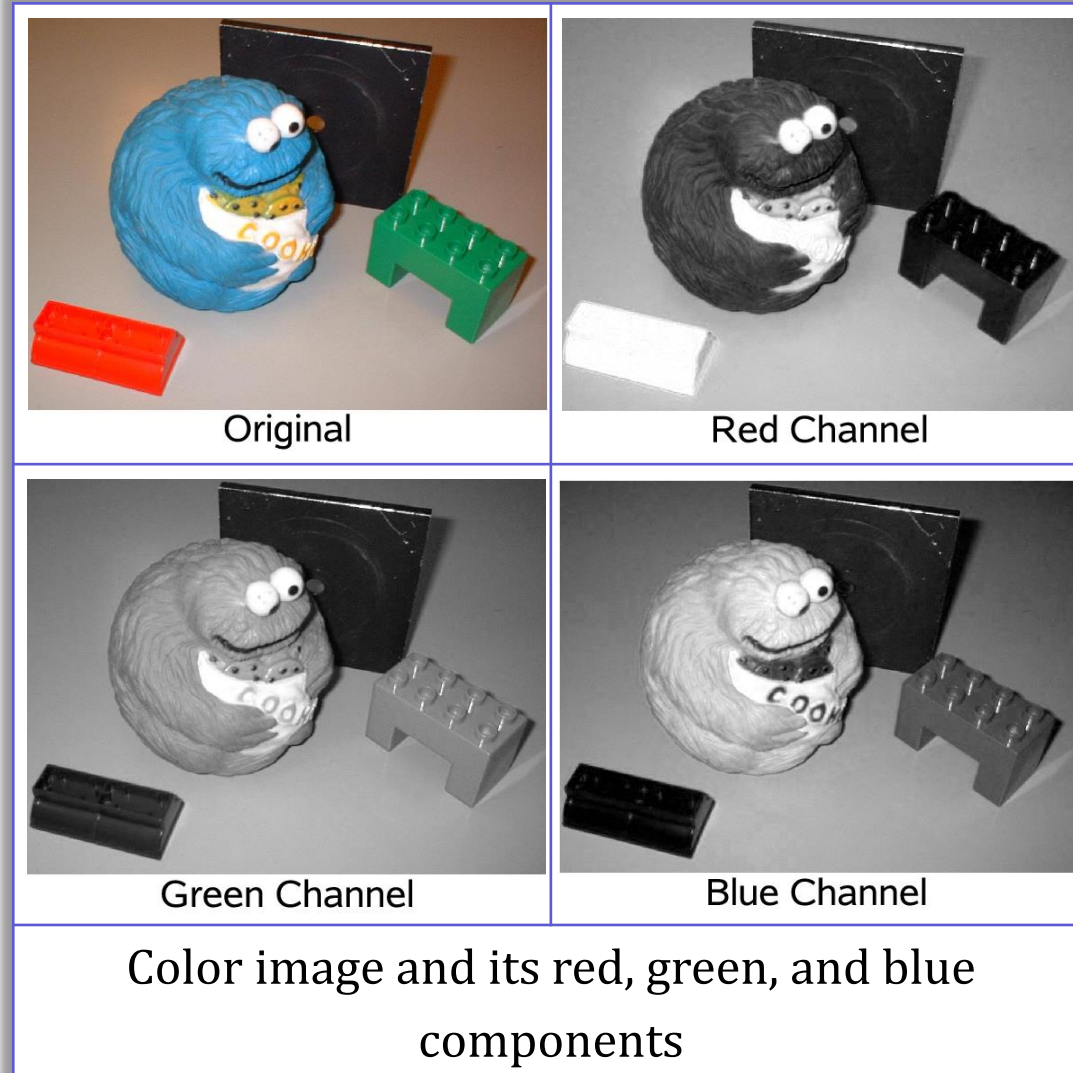
94	0.419607843	0.678431373	0.870588235	<82> R:0.35 G:0.65 B:0.81	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87
95	0.580392157	0.709803922	0.352941176	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87
96	0.482352941	0.678431373	0.776470588	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
97	0.647058824	0.678431373	0.450980392	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
98	0.709803922	0.580392157	0.807843137	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
99	0.580392157	0.709803922	0.611764706	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87
100	0.937254902	0.517647059	0.741176471	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
101	0.580392157	0.709803922	0.741176471	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
102	0.937254902	0.549019608	0.647058824	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
103	0.937254902	0.580392157	0.517647059	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
104	0.776470588	0.741176471	0.290196078	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
105	0	1	1	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
106	0.549019608	0.741176471	0.905882353	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87



An indexed color image and the indices in a 6×6 detailed region. Each pixel shows the index and the values of R, G, and B at the color map entry that the index points to

RGB Color Space

- **Color space** is a structured way of *mapping the real-world colors into numerical values* that can be manipulated, processed, or displayed on digital screens.
- RGB is *the most common color space used for digital image representation* as it conveniently corresponds to the three primary colors which are mixed for display on a monitor or similar device.
- *It can easily separate and view* the red, green and blue components of a truecolor image. The colors in a real image are typically *a blend of color components from all three channels*.



RGB to Greyscale Image Conversion

- An RGB color space can be converted to a greyscale image using a simple transform.
- *Greyscale conversion is the initial step in many image analysis algorithms*, as it essentially simplifies (i.e. reduces) the amount of information in the image.
- Although a greyscale image contains less information than a color image, the majority of important feature related information is maintained such as edges, regions, junctions and so on. Feature detection and processing algorithms then typically operate on the converted greyscale version of the image.



RGB to Greyscale Image Conversion

- An RGB color image, I_{color} , is converted to greyscale image, $I_{greyscale}$, using the following transformation:

$$I_{greyscale}(m,n) = \alpha I_{color}(m,n,r) + \beta I_{color}(m,n,g) + \gamma I_{color}(m,n,b)$$

where (m,n) indexes an individual pixel within the greyscale image and (m,n,c) the individual channel at pixel location (m,n) in the color image for channel c in the red r , green g , and blue b image channels.

- **The greyscale image** is *a weighted sum of the red, green, and blue color channels*. The weighting coefficients (α, β, γ) are set in proportion to the perceptual response of the human eye to each of the red, green and blue color channels.

RGB to Greyscale Image Conversion

- A standardized weighting ensures uniformity (NTSC television standard, $\alpha = 0.2989$, $\beta = 0.5870$, and $\gamma = 0.1140$).
- The human eye is naturally *more sensitive to red and green light*. Thus, these colors are given higher weightings to ensure that the relative intensity balance in the resulting greyscale image is similar to that of the RGB color image.
- RGB to greyscale *conversion is a noninvertible image transform*: the true color information that is lost in the conversion cannot be readily recovered.

Displaying Image Information – MATLAB

- The **imfinfo** function in **MATLAB** can be used to *display information about image files* including their type, format, size and bit depth, etc (without opening them and storing their contents in the workspace).
- The resulting structure (stored in variable `ans`) will contain the following information:

```
imfinfo('Peppers.png');  
  
Filename: 'C:\Program Files\MATLAB\... \peppers.png'  
FileModDate: '16-Dec-2002 03:10:58'  
FileSize: 287677  
Format: 'png'  
Width: 512  
Height: 384  
BitDepth: 24  
ColorType: 'truecolor'  
FormatSignature: [1×8 double]  
InterlaceType: 'none'  
Transparency: 'none'  
ImageModTime: '16 Jul 2002 16:46:41 +0000'  
Description: 'Zesty peppers'  
Copyright: 'Copyright The MathWorks, Inc.'
```


Reading Images – MATLAB

- **MATLAB** has a built-in function called **imread** that can be used to *open and read the contents of image files* in most common formats (e.g., TIFF, JPEG, BMP, GIF, and PNG).
- The imread function creates/exports the appropriate 2D/3D image arrays within the MATLAB environment via storing them into a variable in the MATLAB workspace.

```
Img1 = imread('cameraman.tif');  
Img2 = imread('Peppers.png');  
Img3 = imread('coins.png');
```

Workspace	
Name ▲	Value
 Img1	256x256 uint8
 Img2	384x512x3 uint8
 Img3	246x300 uint8

Reading Images – MATLAB

- In **MATLAB**, to read in an indexed image, we must specify variables for both the image and its color map.

```
[Img,Map] = imread('trees.tif');
```

Workspace	
Name ▲	Value
Img	258x350 uint8
Map	256x3 double

Writing Images – MATLAB

- **MATLAB** has a built-in function called **imwrite** that can be used to *write the contents of an image* in one of the most common graphic file formats.
- If the output file format uses *lossy compression* (e.g., JPEG), imwrite allows the specification of a *quality parameter*, which controls the trade-off between the resulting image's quality and the file size.
- In **MATLAB**, the image written to file in *a lossy compression format* (e.g. JPEG) will not be stored as the exact MATLAB image representation it started as. *Image pixel values will be altered* in the image output process as a result of the lossy compression. *This is not the case if a lossless compression technique is employed.*

Writing Images – MATLAB

- **Example:** Read an image from a PNG file and save it to a JPG file using three different quality parameters: 75 (default), 5 (poor quality, small size), and 95 (better quality, larger size).

```
Img1 = imread('Peppers.png');  
imwrite(Img1, 'PeppersQuality_75.jpg');  
imwrite(Img1, 'PeppersQuality_05.jpg', 'quality', 5);  
imwrite(Img1, 'PeppersQuality_95.jpg', 'quality', 95);
```



Original image



Compressed image (JPG, q = 75, file size = 24 kB)



Compressed image (JPG, q = 5, file size = 8 kB)



Compressed image (JPG, q = 95, file size = 60 kB)

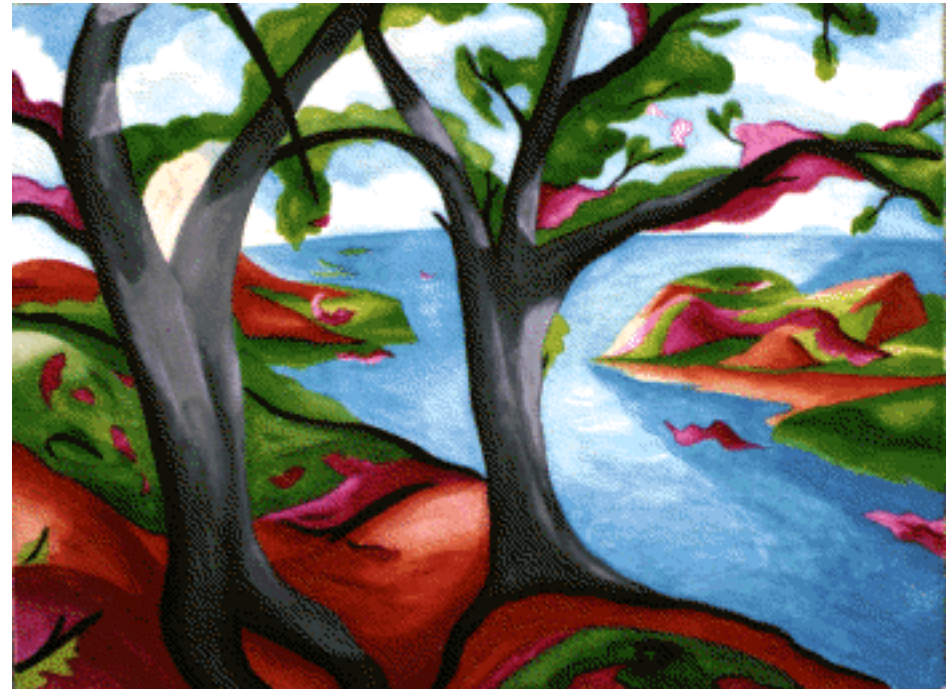
Writing Images – MATLAB

- To write the contents of an *indexed image*, we must pass both *the image and its color map* to `imwrite` function.

```
imwrite(Img, 'TreesOriginal.tif');
```



```
imwrite(Img,Map, 'TreesOriginal.tif');
```



Basic Display of Images – MATLAB

■ **MATLAB** has several functions for displaying images:

- **image**: displays an image using the current color map. The color map array is an $M \times 3$ matrix of class double, where each element is a floating-point value in the range $[0, 1]$. Each row in the color map represents the R, G, and B values for that particular row.
- **imagesc**: accepts input arrays of any MATLAB storage type (uint8, uint16 or double) and any numerical range. It then *scales image data to the full range of the current/default color map and displays the image*.
- **imshow**: displays an image and contains a number of optimizations and optional parameters for property settings associated with image display.
- **imtool**: displays an image and contains a number of associated tools that can be used to explore the image contents.

Basic Display of Images – MATLAB

- Displaying intensity images using different **imshow** options:

(a) Opens a new figure and displays the image in its original state.

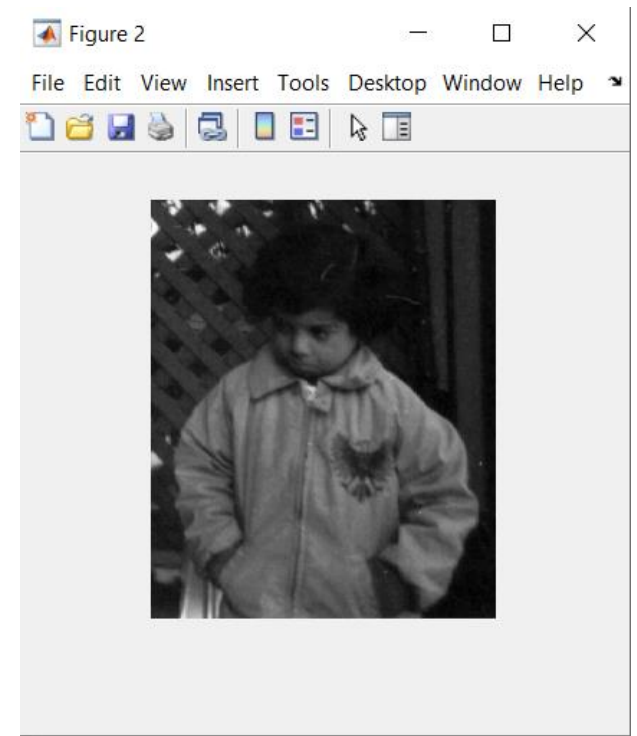
(b) Opens a new figure and displays a scaled version of the same image.

```
Img = imread('pout.tif');  
figure, imshow(Img)
```



(a)

```
Img = imread('pout.tif');  
figure, imshow(Img, [])
```



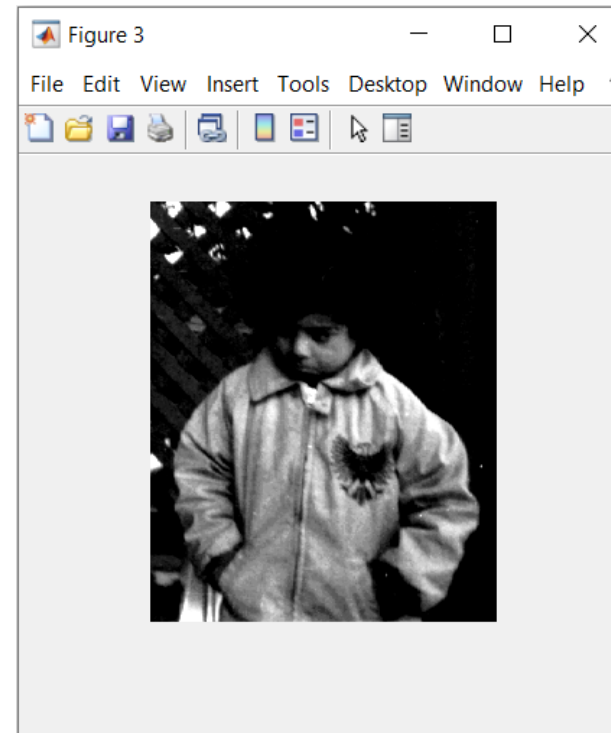
(b)

Basic Display of Images – MATLAB

- Displaying intensity images using different **imshow** options:

(c) Opens a new figure and scales image display using the [min max] range. It specifies a range of gray levels, such that all values lower than 100 will be displayed as black and all values greater than 160 will be displayed as white.

```
Img = imread('pout.tif');  
figure, imshow(Img, [100 160])
```

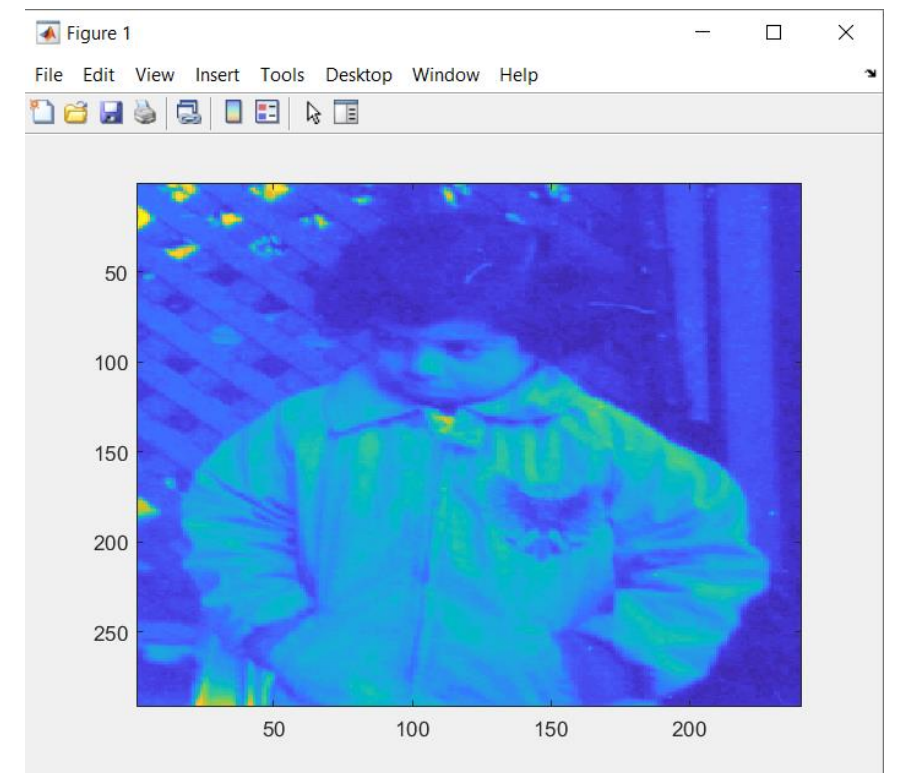


(c)

Basic Display of Images – MATLAB

- Displaying intensity (greyscale) image using **imagesc** function:
- In this case, additional steps are required when using **imagesc** function to display conventional images.
- The `imagesc` function uses the current colormap, which is usually parula.
- We can control the current/default color map using the **colormap** function in **MATLAB**.

```
Img = imread('pout.tif');  
figure, imagesc(Img)
```

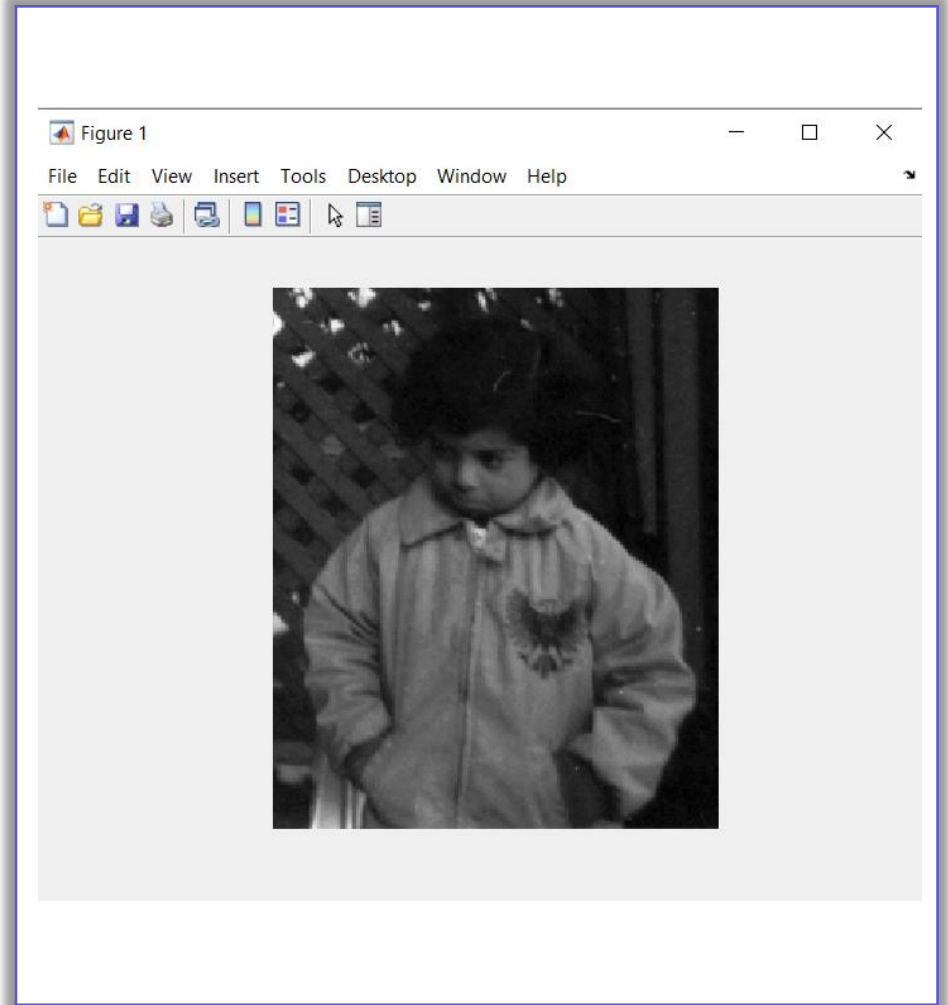


Basic Display of Images – MATLAB

- Displaying intensity (greyscale) image using **imagesc**

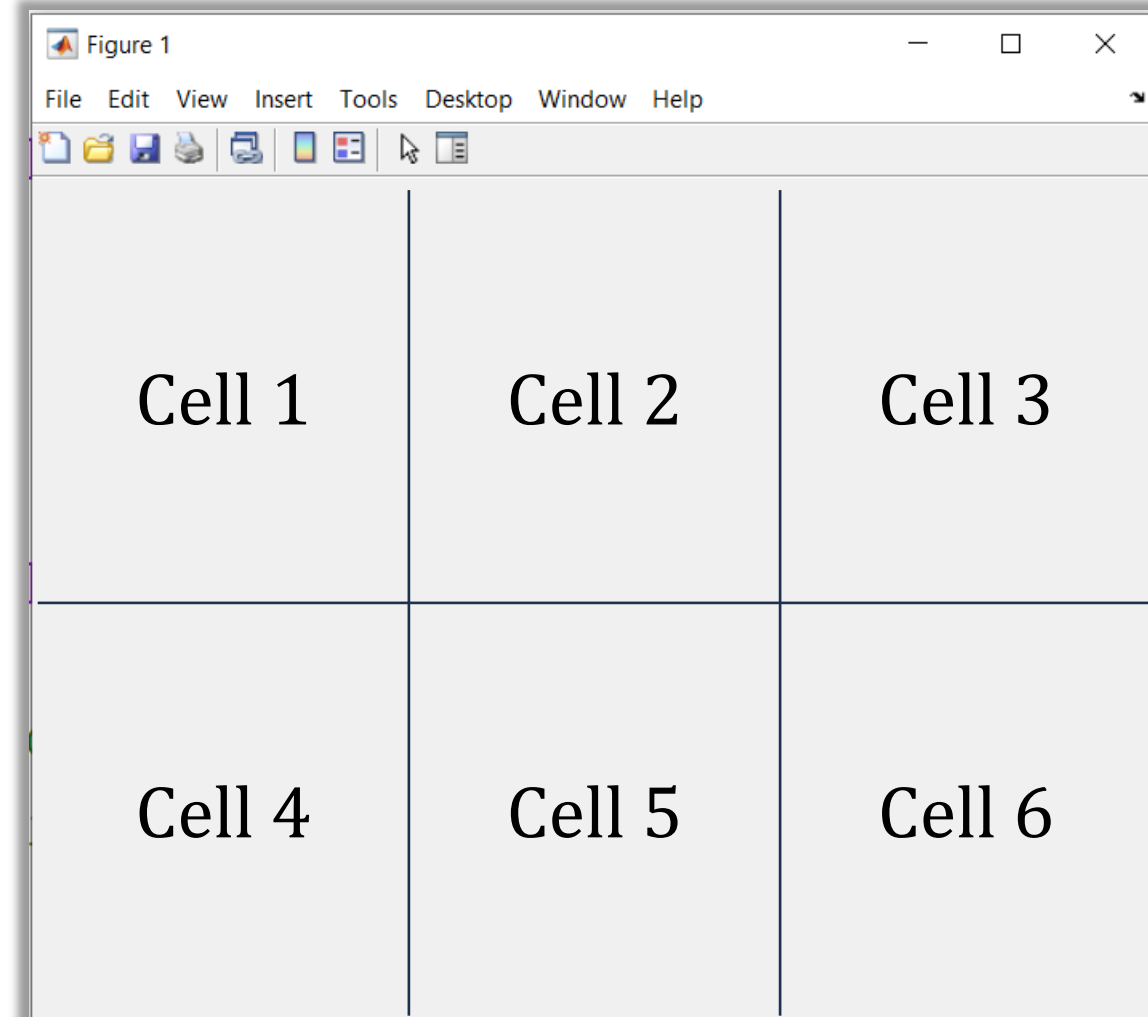
function:

```
Img = imread('pout.tif');  
figure, imagesc(Img)  
axis image; % Correct aspect ratio of displayed image  
axis off; % Turn off the axis labelling  
colormap(gray) % Display intensity image in grey scale
```



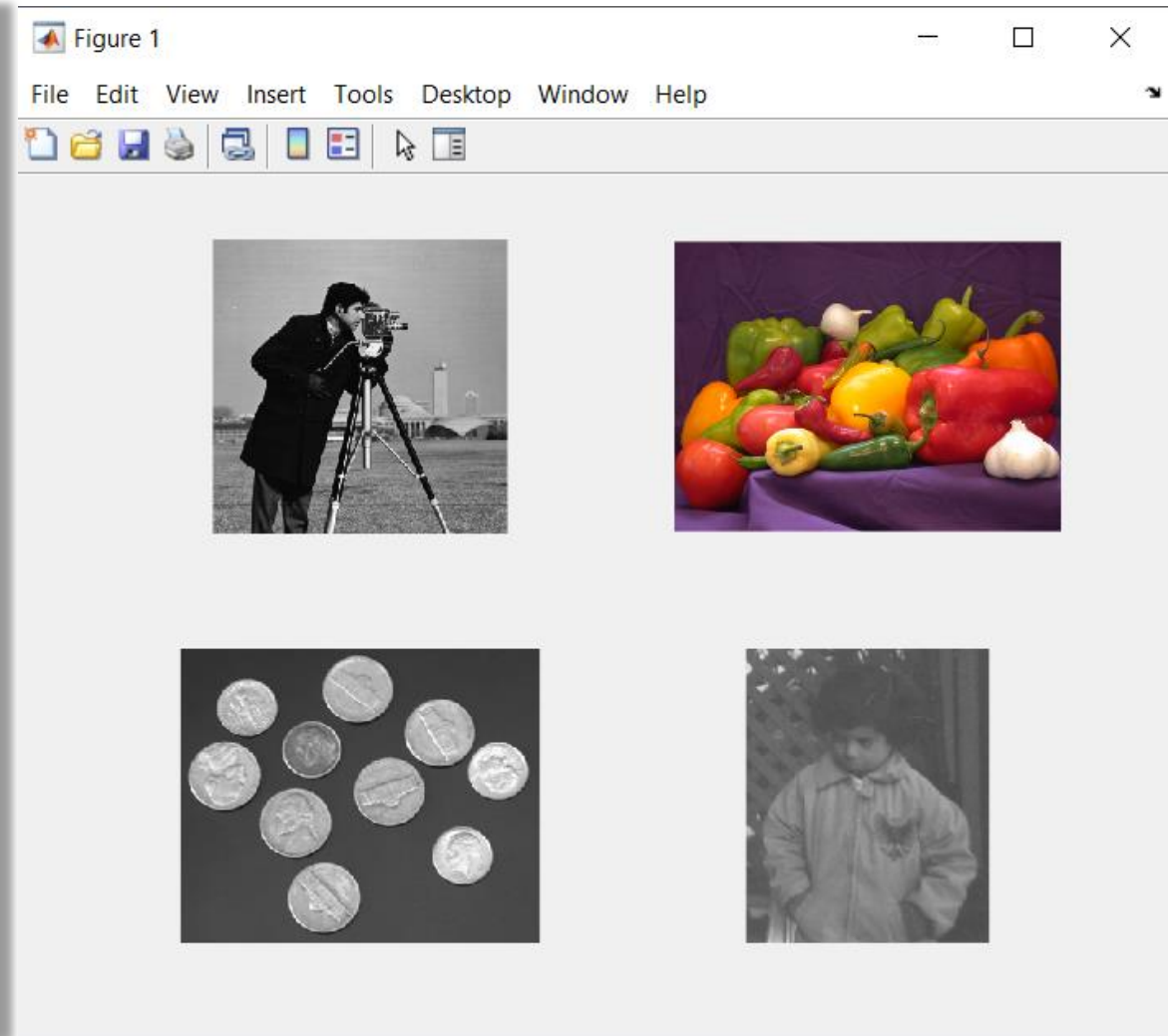
Basic Display of Images – MATLAB

- **MATLAB** has a built-in function called **subplot** that can be used to display multiple images within one figure.
- When using this function, the first two parameters specify the number of rows and columns to divide the figure. The third parameter specifies which subdivision to use.
- In the case of **subplot(2, 3, 3)**, we are telling **MATLAB** to divide the figure into two rows and three columns and set the third cell as active.



Basic Display of Images – MATLAB

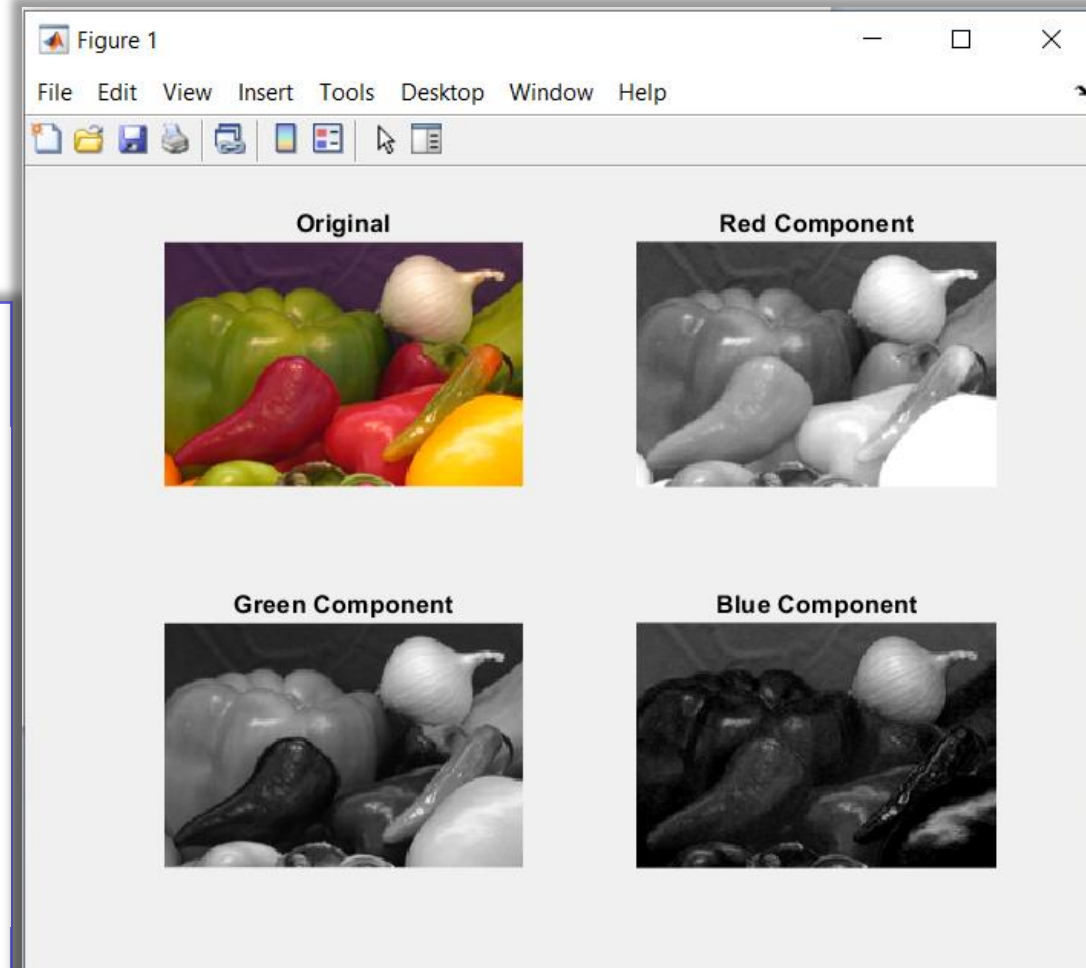
```
Img1 = imread('cameraman.tif');  
Img2 = imread('Peppers.png');  
Img3 = imread('Coins.png');  
Img4 = imread('Pout.tif');  
figure  
subplot(2,2,1), imshow(Img1)  
subplot(2,2,2), imshow(Img2)  
subplot(2,2,3), imshow(Img3)  
subplot(2,2,4), imshow(Img4)
```



Basic Display of Images – MATLAB

- We can access individual channels of an RGB image and extract them as separate images in their own right.

```
Img = imread('Onion.png');%Read in 8 bit RGB color image
ImgRed = Img(:,:,1);%Extract red channel (first channel)
ImgGreen = Img(:,:,2);%Extract green channel (second channel)
ImgBlue = Img(:,:,3);%Extract blue channel (third channel)
figure
subplot(2,2,1),imshow(Img);title('Original');
subplot(2,2,2),imshow(ImgRed);title('Red Component');
subplot(2,2,3),imshow(ImgGreen);title('Green Component');
subplot(2,2,4),imshow(ImgBlue);title('Blue Component');
```

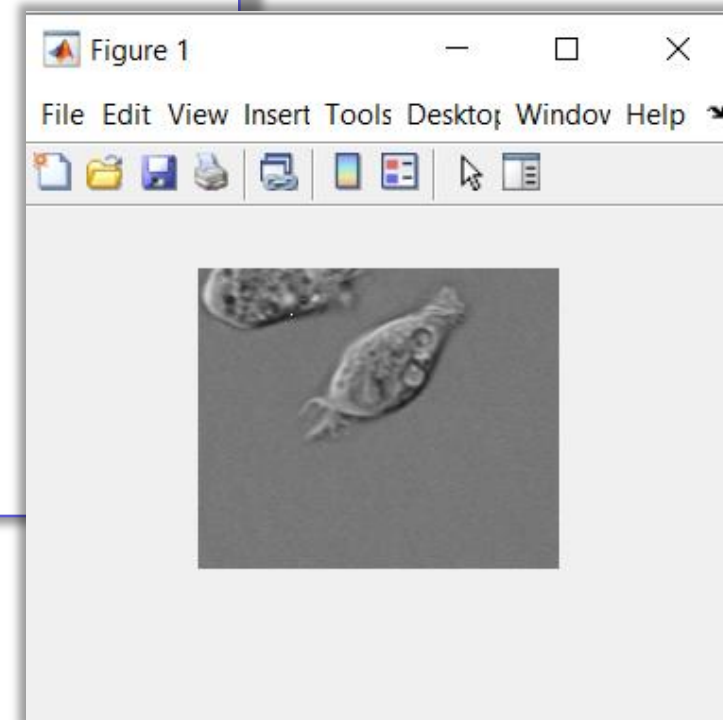


Accessing Pixel Values – MATLAB

■ We can *access individual pixel values* within the image and change their value.

■ **Example 1:**

```
Img1 = imread('cell.tif'); %Read in 8 bit intensity image of cell
Img1(25,50) %Print pixel value at location (25,50)
ans =
    uint8
        46
Img1(25,50) = 255; %Set pixel value at (25,50) to white
imshow(Img1); %View resulting changes in image
```

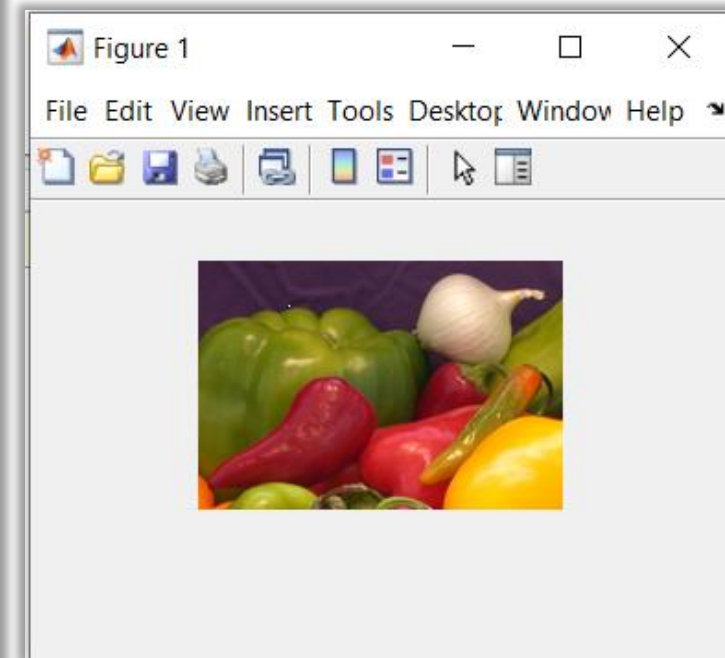


Accessing Pixel Values – MATLAB

- We can *access individual pixel values* within the image and change their value.

- **Example 2:**

```
Img2 = imread('onion.png'); %Read in 8 bit color image
Img2(25,50,:) %Print RGB pixel value at location (25,50)
    1×1×3 uint8 array
    ans(:, :, 1) = 46
    ans(:, :, 2) = 29
    ans(:, :, 3) = 50
Img2(25,50, 1) %Print only the red value at (25,50)
    ans =
        uint8
        46
Img2(25,50,:) = [255, 255, 255]; %Set pixel value to RGB white
imshow(Img2);
```



Converting Image Types – MATLAB

- The most common data classes for images in **Matlab** are as follows:
 - **uint8:** An image can be stored as a uint8 data type, where each pixel is represented by 1 byte, usually in the [0, 255] range.
 - **uint16:** An image can be stored as a uint16 data type, where each pixel is represented by 2 bytes, usually in the [0, 65535] range.
 - **double:** An image can be stored as a double data type (floating-point representation), where each pixel is represented by 8 bytes, usually in the [0.0, 1.0] range. It is commonly used for precise calculations in image processing.
 - **logical:** An image can be stored as a logical data type, where each pixel is represented by a single bit (1 byte per pixel in memory alignment). This format is commonly used for binary (black and white) images in image processing.

Converting Image Types – MATLAB

■ Example:

```
Img = imread('Peppers.png'); %Img is an RGB image of type uint8  
ImgDouble = im2double(Img); %Convert Img to double type
```

Workspace	
Name ▲	Value
 Img	384x512x3 uint8
 ImgDouble	384x512x3 double

- We can verify that the new image consists of pixel values in the range [0, 1].

```
min(ImgDouble(:))  
ans = 0
```

```
max(ImgDouble(:))  
ans = 1
```

Converting Image Types – MATLAB

■ Example:

```
Img = imread('Peppers.png'); %Img is an RGB image of type uint8
ImgUint16 = im2uint16(Img); %Convert Img to uint16 type
```

Workspace	
Name ▲	Value
 Img	384x512x3 uint8
 ImgUint16	384x512x3 uint16

- We can verify that the new image consists of pixel values in the range [0, 65535].

```
min(ImgUint16(:))
ans =
    uint16
     0
```

```
max(ImgUint16(:))
ans =
    uint16
   65535
```

Converting Image Types – MATLAB

- **MATLAB** also has a built-in function called **im2bw** that can be used to convert images to binary equivalents.

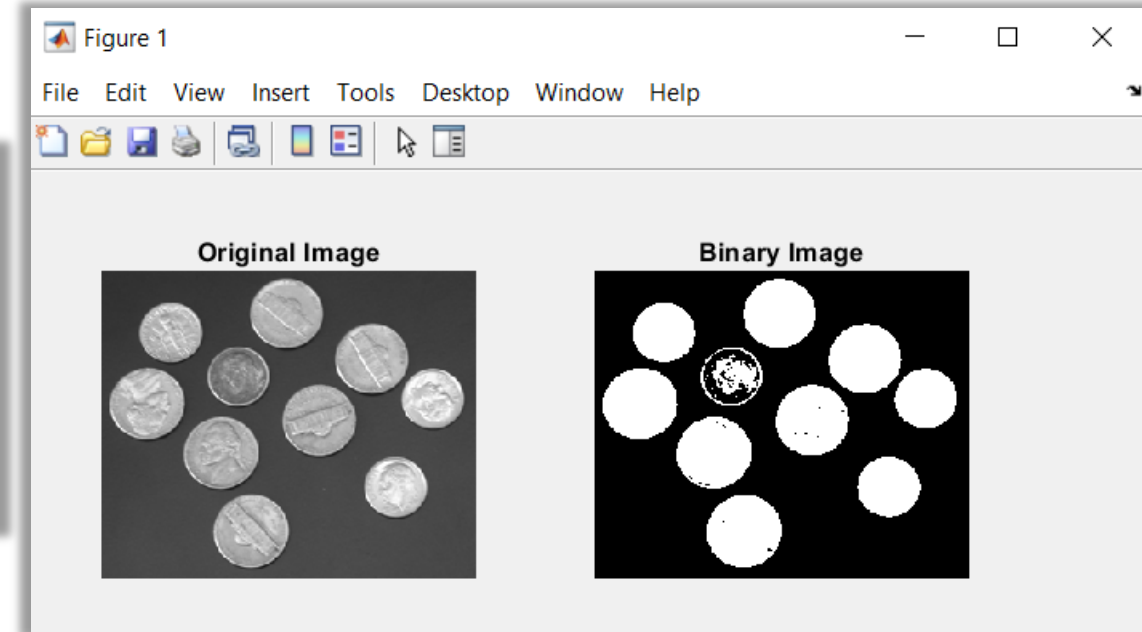
```
ImgB = im2bw( Img , Level );  
ImgB = im2bw( IndexedImg , Map , Level );
```

- **im2bw** function converts the grayscale/color image or indexed image to binary image, by replacing all pixels in the input image with luminance greater than **Level** with the value 1 (white) and replacing all other pixels with the value 0 (black). **Level** is a luminance threshold which is a *nonnegative number between 0 and 1* and the default value is 0.5.

Converting Image Types – MATLAB

■ Example:

```
Img = imread('Coins.png');  
ImgB = im2bw(Img);  
figure  
subplot(1,2,1),imshow(Img);title('Original Image');  
subplot(1,2,2),imshow(ImgB);title('Binary Image');
```



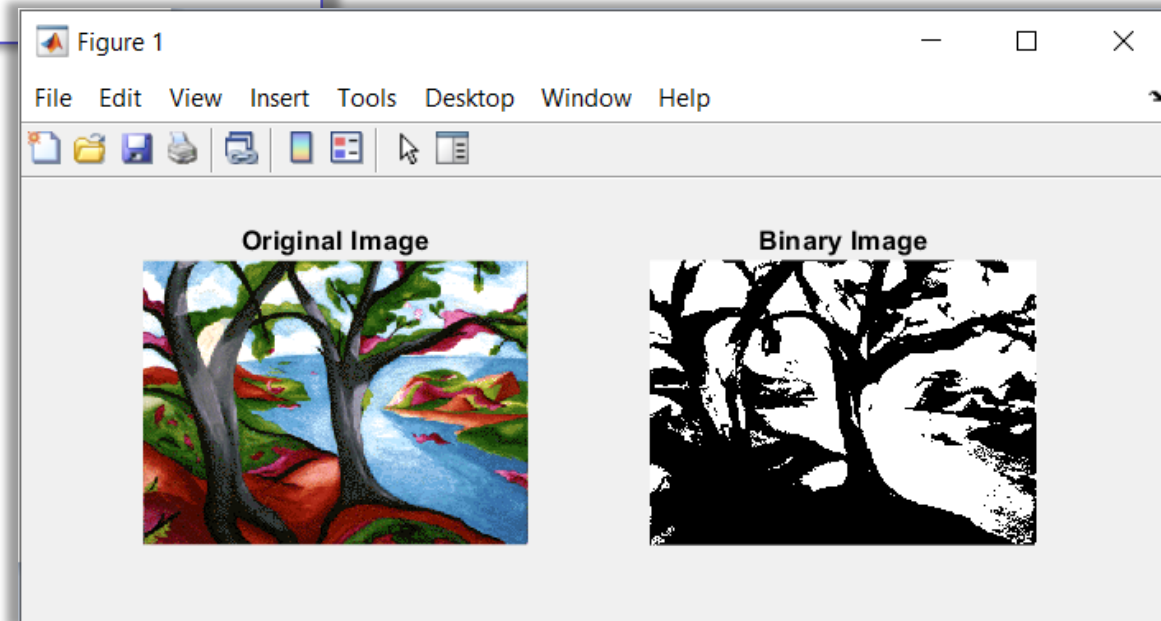
- Any value greater than 0.5 becomes 1 (true), otherwise it becomes 0 (false). Note that since `im2bw` expects a threshold luminance level as a parameter, and requires it to be a nonnegative number between 0 and 1, it implicitly assumes that the input variable (`Img`) is a *normalized grayscale image of class double*.

Converting Image Types – MATLAB

■ Example:

```
[Img,Map] = imread('trees.tif');  
ImgB = im2bw(Img,Map,0.4);  
figure  
subplot(1,2,1),imshow(Img,Map);title('Original Image');  
subplot(1,2,2),imshow(ImgB);title('Binary Image');
```

- Any value greater than 0.4 becomes 1 (true), otherwise, it becomes 0 (false).



Converting Image Types – MATLAB

- Some operations may require you to convert an image from one type to another.
- **MATLAB** also includes functions to convert between RGB (truecolor), indexed image, and grayscale image.

Name	Converts	Into
ind2gray	An indexed image	Its grayscale equivalent
gray2ind	A grayscale image	An indexed representation
rgb2gray	An RGB (truecolor) image	Its grayscale equivalent
rgb2ind	An RGB (truecolor) image	An indexed representation
ind2rgb	An indexed color image	Its RGB (truecolor) equivalent

Converting Image Types – MATLAB

- We can also convert *RGB image to a grayscale image* as follows:

```
GrayImg = rgb2gray( RGBImg );
```

- We can convert *RGB image to an indexed image* as follows:

```
[IndexedImg , Map] = rgb2ind( RGBImg , Q );
```

where **Q** denotes the number of quantized colors used for minimum variance quantization, specified as a positive integer that is less than or equal to 65,536. The returned colormap **Map** has **Q** or fewer colors.

Converting Image Types – MATLAB

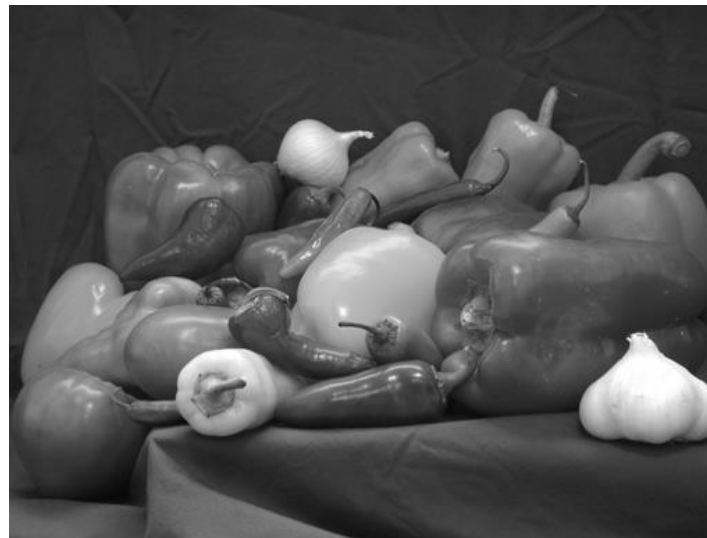
■ Example:

```
RGBImg = imread('Peppers.png'); %Read in an RGB image  
GrayImg = rgb2gray(RGBImg); %Convert RGB image to grayscale image  
IndexedImg = rgb2ind(RGBImg,50); %Convert RGB image to indexed image
```

RGB Image



Greyscale Image



**Indexed Image
with 50 quantized colors**

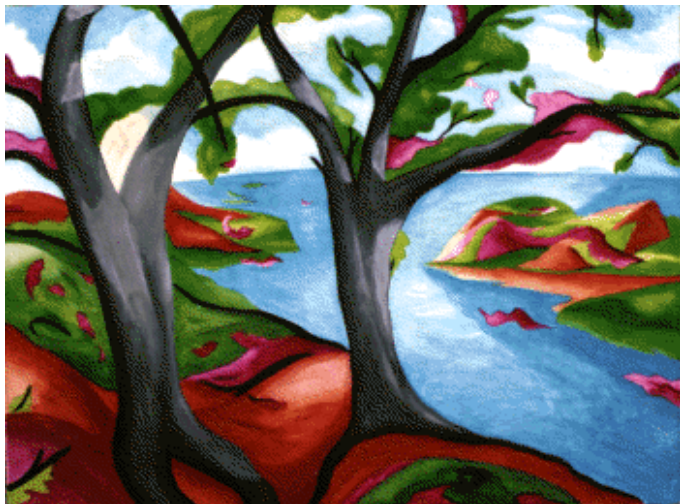


Converting Image Types – MATLAB

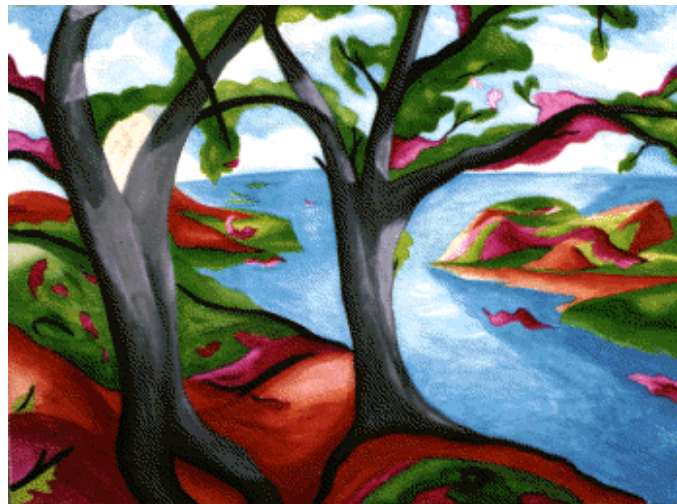
■ Example:

```
[IndexedImg,Map] = imread('trees.tif');%Read in an indexed image  
RGBImg = ind2rgb(IndexedImg,Map);%Convert an indexed image to RGB image  
GrayImg = ind2gray(IndexedImg,Map);%Convert an indexed image to intensity image
```

Indexed Image



RGB Image



Greyscale Image



Exploring Contents of Images – MATLAB

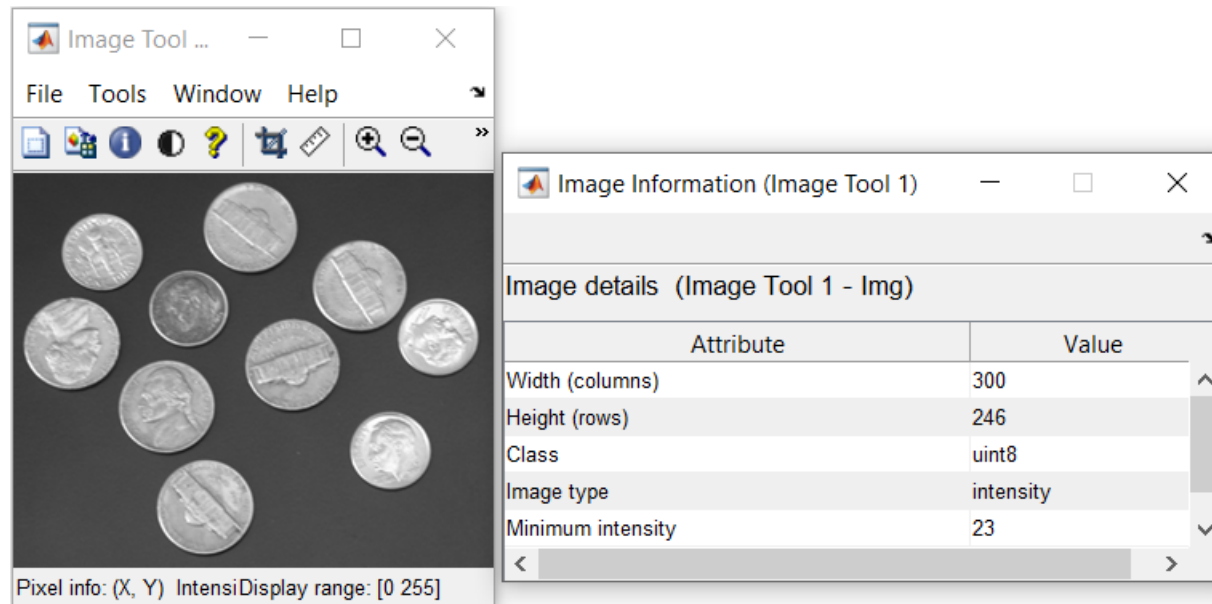
- MATLAB has a built-in function called **imtool** that can be used to *inspect the contents of an image more closely*.
- This function provides all the image display capabilities of **imshow** as well as access to other tools for navigating and exploring images, such as the *Pixel Region tool*, *Image Information tool*, and *Adjust Contrast tool*.
- These tools can also be directly accessed using the **imshow** function with their library functions **impixelinfo**, **imageinfo**, and **imcontrast**, respectively.

Exploring Contents of Images – MATLAB

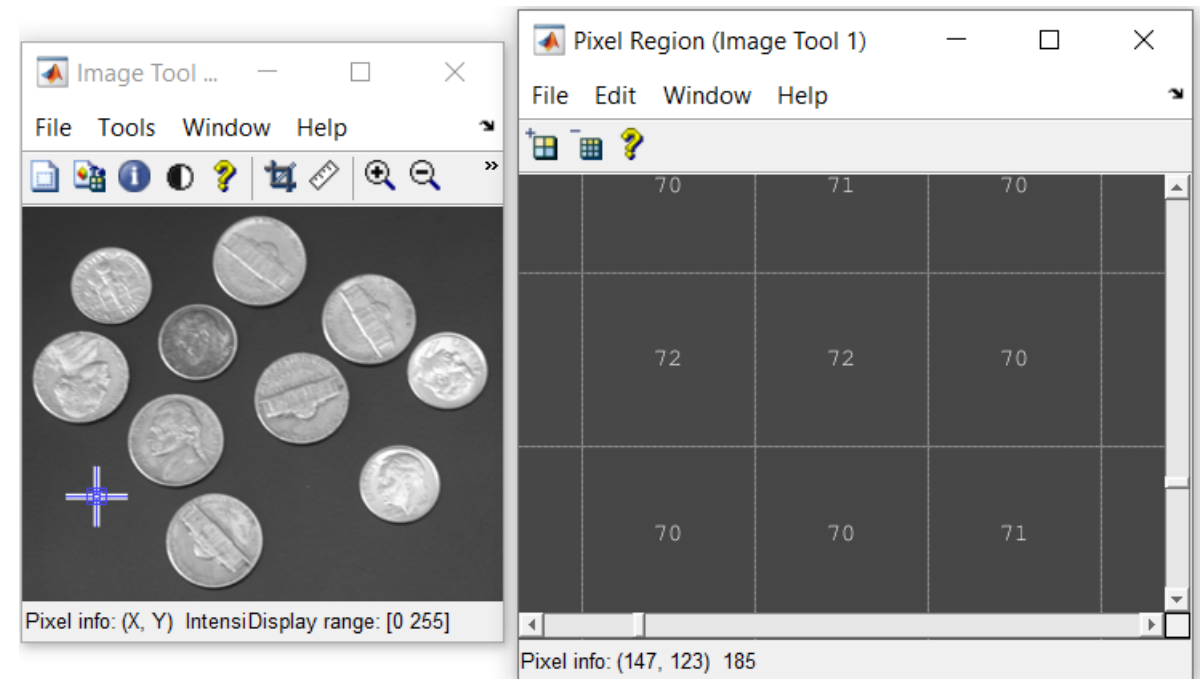
■ Example:

```
Img = imread('Coins.png');  
imtool(Img)
```

Image Information Tool

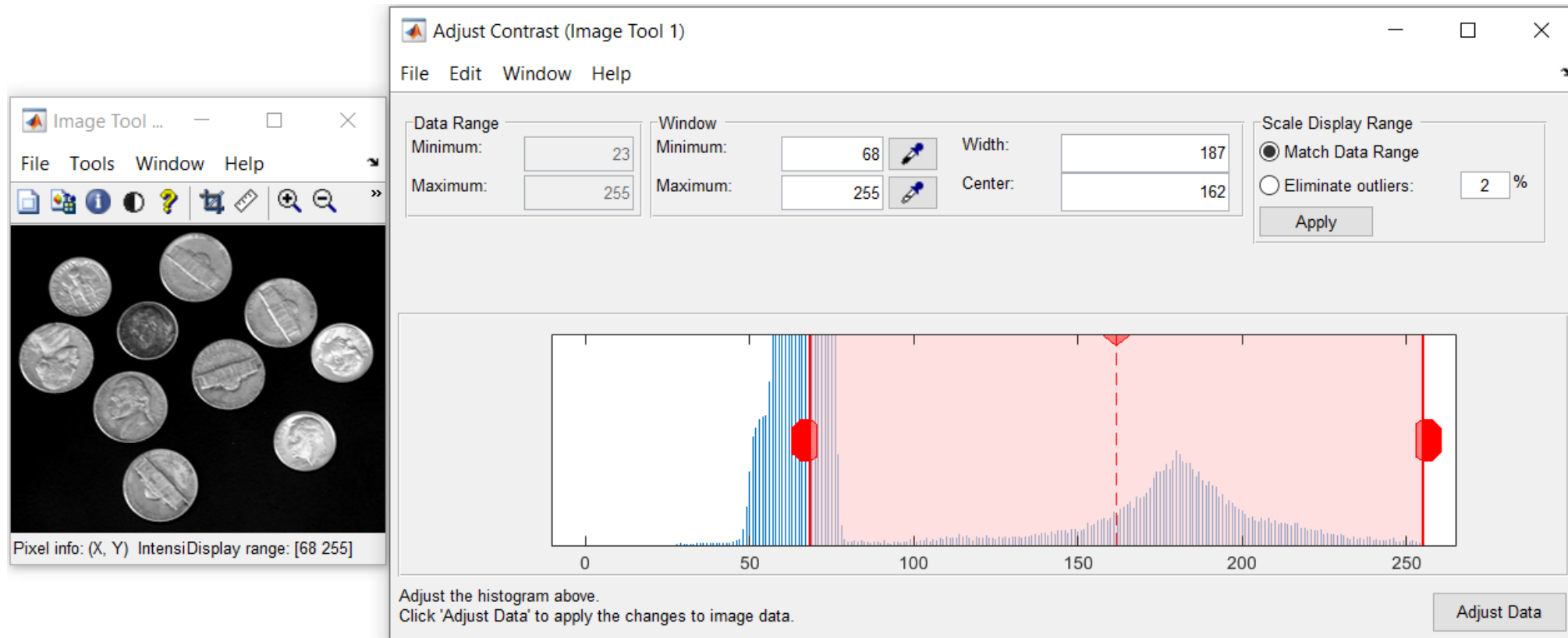


Pixel Region Tool



Exploring Contents of Images – MATLAB

Adjust Contrast Tool

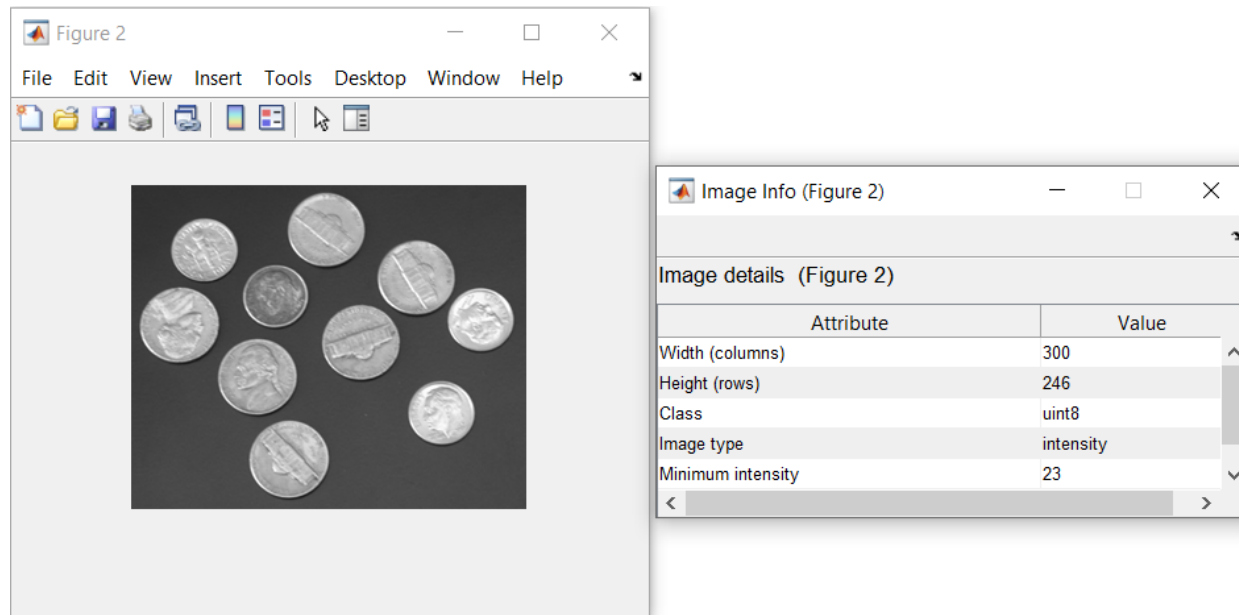


Exploring Contents of Images – MATLAB

■ Example:

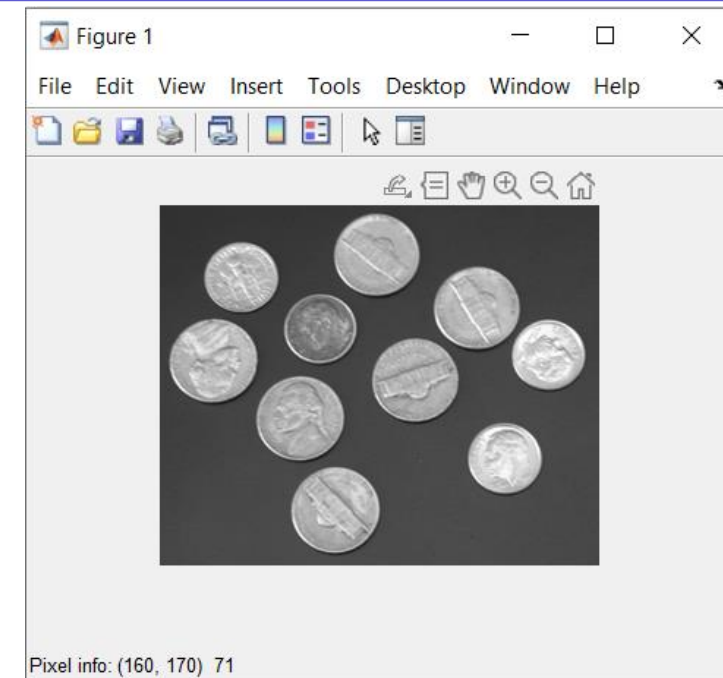
Image Information Tool

```
Img = imread('Coins.png');  
figure, imshow(Img), imageinfo
```



Pixel Region Tool

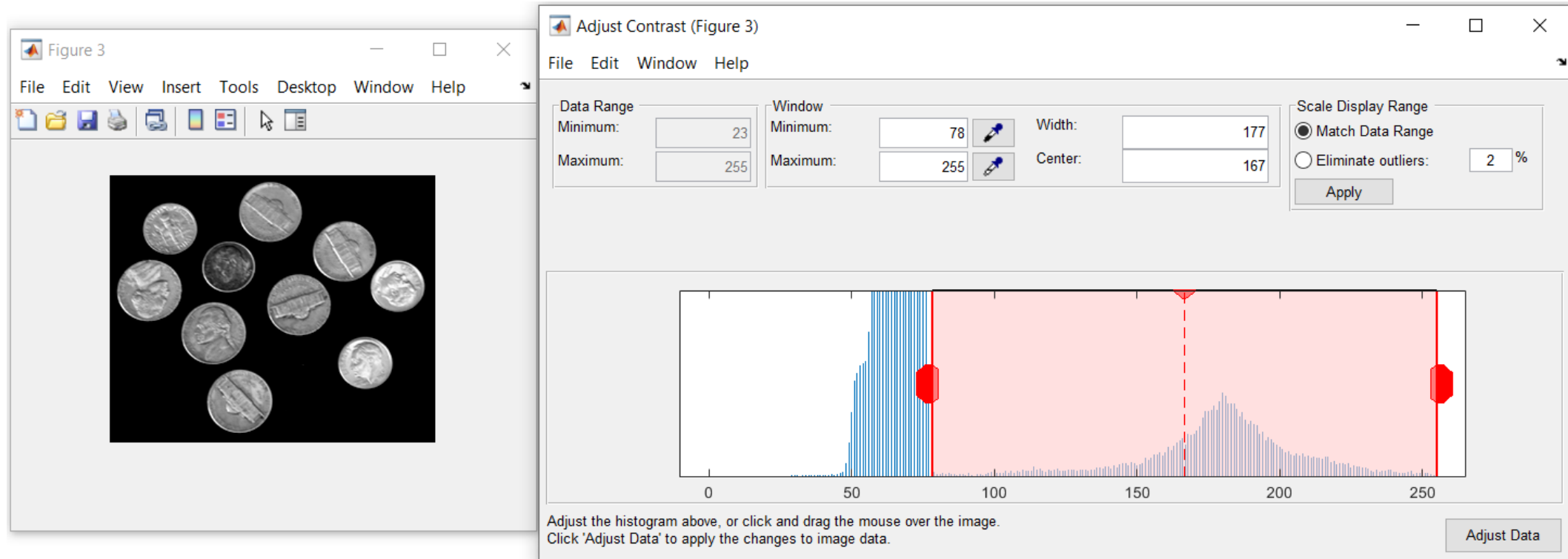
```
Img = imread('Coins.png');  
figure, imshow(Img), impixelinfo
```



Exploring Contents of Images – MATLAB

Adjust Contrast Tool

```
Img = imread('Coins.png');  
figure, imshow(Img), imcontrast
```



Thank
You